

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY

Proyecto MiBici: propuesta y base reproducible

Equipo 34

Diego Falcón Costilla A01139580

Oscar Elton A01840474

Mariana Favarony Avila A01704671

Javier Alejandro Garcia Flores A01797672

Operaciones de aprendizaje automático (TC5061)

1. Repositorio y alcance

Repositorio del equipo: github.com/diegofalconcostilla/TC5061-Proyecto-Equipo34. Contiene la estructura generada con Cookiecutter Data Science, la ingesta reproducible y versionada de los viajes de MiBici (`mibici/dataset.py`), el módulo de feature engineering (`mibici/features.py`), el experimento base en MLflow (`mibici/modeling/train.py`), un notebook de exploración, el ML Canvas y el diagrama de arquitectura en `references/`. Con tres comandos documentados en el README se reconstruyen los datos y se repiten los doce runs del experimento. Este documento presenta la propuesta de solución para las tres preguntas del proyecto y los resultados de la línea base contra la que se medirán las siguientes entregas.

2. Formulación del problema

La Figura 1 resume el ML Canvas (Dorard, 2018). La propuesta de valor es pasar de una operación reactiva a una anticipada: saber cuánta demanda tendrá cada estación, cuáles se van a vaciar o llenar en las próximas horas y qué zonas de la ciudad se comportan de forma parecida.

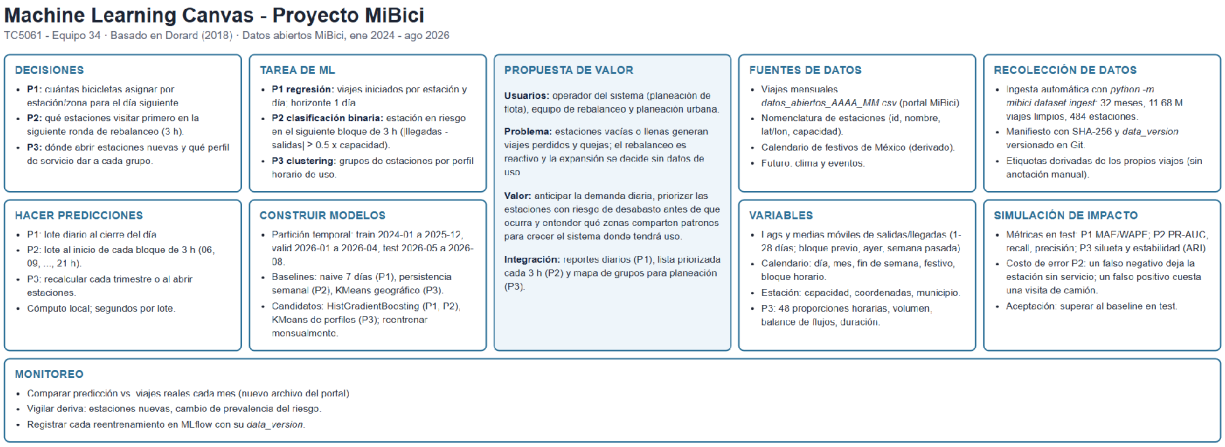


Figura 1. ML Canvas del proyecto MiBici.

Tabla 1. Formulación de las tres preguntas como problemas de ML.

	Qué se predice o agrupa	**Granularidad y horizonte**	**Relevancia**	**Usuario y decisión**
P1 Regresión	Número de viajes iniciados en una estación en un día.	Estación x día; se predice el día *d* con información hasta el cierre de *d*-1.	La demanda diaria por estación define cuántas bicicletas debe haber en cada punto al iniciar el día.	Operador del sistema: distribuir la flota y programar mantenimiento en estaciones de baja demanda.
P2 Clasificación	Si una estación estará en riesgo de quedarse sin bicicletas o sin anclajes.	Estación x bloque de 3 h (06-09, ..., 21-24); se predice el bloque siguiente al inicio de cada bloque.	Una estación vacía o llena es un viaje perdido o un usuario que no puede devolver la bicicleta.	Equipo de rebalanceo: lista priorizada de estaciones a visitar en la siguiente ronda.
P3 Clustering	Grupos de estaciones con patrones de uso similares y su ubicación.	Estación; perfil calculado con 2024-2025 (periodo de entrenamiento).	Revela tipos de zona (residencial, laboral, recreativa) que no se ven en la ubicación.	Planeación urbana: dónde y con qué perfil de servicio expandir el sistema.

Aproximación del riesgo en P2. MiBici no publica el inventario de bicicletas por estación, pero sí cada viaje con su estación de origen y destino. Para cada estación y bloque se calcula el flujo neto `llegadas - salidas`. Se etiqueta `risk_empty` si salen al menos ⌈0.5 x capacidad⌉ bicicletas más de las que llegan y `risk_full` si llegan esa cantidad de más; `risk` es la unión de ambas. El razonamiento es que, partiendo de una ocupación típica cercana a la mitad de los anclajes, un desbalance de media capacidad en 3 horas basta para vaciar o llenar la estación si nadie rebalancea. Con este umbral el 5.0 % de los bloques de entrenamiento queda en riesgo: un evento poco frecuente pero con suficientes positivos para aprender. Un umbral de 0.3 marcaría 13.8 % de los bloques, incluyendo desbalances que el sistema absorbe normalmente, y uno de 0.75 dejaría solo 1.1 %. El riesgo se concentra en los bloques de 06-09 h y 15-21 h, lo que coincide con los traslados laborales.

3. Definición de los datos

Rango de fechas. Se usan los viajes de enero de 2024 a agosto de 2026 (32 meses). Dos años completos de entrenamiento capturan dos veces cada efecto anual (Semana Santa, vacaciones de verano, diciembre) y dejan ocho meses recientes para validar y probar. Se descartan los años anteriores porque el sistema ha cambiado mucho (estaciones nuevas y retiradas, tarifas), así que datos más viejos describen otro sistema y multiplicarían el volumen sin aportar información vigente.

1) Fuente. Portal de datos abiertos de MiBici (mibici.net/es/datos-abiertos): un archivo `datos_abiertos_AAAA_MM.csv` por mes y la nomenclatura `nomenclatura_2026_09.csv`. Cada URL incluye un identificador numérico que no se puede inferir (por ejemplo `/site/assets/files/130362/datos_abiertos_2026_08.csv`), por lo que la ingesta lee los enlaces de la página y los fija en el manifiesto.

2) Tamaño real obtenido.

Tabla 2. Volumen de datos tras la ingesta (`data_version = efa3a16661ec`).

Concepto	Valor	Concepto	Valor
Meses	32	Estaciones en catálogo	484
Viajes crudos	12,049,962	Estaciones con viajes en el rango	473
Viajes descartados (duración < 1 o > 180 min)	372,246 (3.1 %)	Archivos en Latin-1 / UTF-8	15 / 18
Viajes limpios	11,677,716	CSV crudos / Parquet limpio	981 MB / 172 MB

3) Variables originales y variables objetivo. De los viajes se usan `Viaje_Id`, `Inicio_del_viaje`, `Fin_del_viaje`, `Origen_Id` y `Destino_Id`; de la nomenclatura, `id`, `name`, `latitude`, `longitude` y `dpcapacity`. El objetivo de P1 es el conteo de viajes con `Origen_Id` = estación y fecha de inicio = día. El objetivo de P2 se construye sumando salidas (por hora de inicio) y llegadas (por hora de fin) en cada bloque de 3 h y aplicando la regla de la sección 2 con la capacidad de la estación. P3 no tiene objetivo: el vector de cada estación son 48 proporciones horarias de salidas (24 de días hábiles y 24 de fin de semana), el volumen diario medio en escala logarítmica, la razón de balance (llegadas - salidas)/(llegadas + salidas), la proporción de uso en fin de semana y la duración mediana.

4) Condiciones de uso y datos personales. El portal publica los archivos como datos abiertos con la invitación "¡Explora, experimenta y comparte los datos de uso de Mi Bici!", pero no declara una licencia explícita: el sitio indica "Todos los derechos reservados Mi Bici © 2026" y el enlace al "Manual para uso de datos abiertos" no lleva a un manual. En consecuencia, el equipo usa los datos solo con fines académicos, cita la fuente, no redistribuye los archivos (el repositorio no contiene datos; cada quien los descarga del portal con la ingesta) y sigue el aviso de privacidad del sistema, que se rige por la Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados (MiBici, s.f.-a, s.f.-b). Los viajes incluyen `Usuario_Id`, `Genero` y

`Año_de_nacimiento`. Ninguna de las tres preguntas los necesita y, combinados con la hora y las estaciones de cada viaje, podrían ayudar a reidentificar personas, así que la ingesta los elimina antes de escribir cualquier archivo limpio. No aparecen en los datasets de modelado, en el notebook, en los parámetros, etiquetas ni artefactos de MLflow.

5) Partición. La partición es temporal y está centralizada en `config.py`: entrenamiento de enero de 2024 a diciembre de 2025, validación de enero a abril de 2026 y prueba de mayo a agosto de 2026. Una partición aleatoria pondría en entrenamiento días posteriores a los de prueba y el modelo aprendería del futuro (Huyen, 2022). Además, todas las variables se calculan con información disponible antes del periodo que se predice: los lags usan desplazamientos de al menos un periodo, las medias móviles se aplican sobre la serie ya desplazada (`add_lag_features` rechaza un lag menor al horizonte y hay pruebas automáticas que lo verifican), el umbral de decisión de P2 se elige en validación y no en prueba, y los perfiles y el escalamiento de P3 se ajustan solo con entrenamiento. La validación y la prueba contienen unas 100 estaciones que abrieron en 2026 (471 estaciones en prueba contra 373 en entrenamiento); por eso los modelos usan atributos de la estación y lags cortos, no un identificador de estación, y así pueden predecir para estaciones nuevas.

4. Pipeline de Machine Learning

Las tres preguntas comparten las primeras etapas y se separan en la agregación:

- **Ingesta y limpieza** (`dataset.py`): descarga, detección de codificación, mapeo de encabezados por palabra clave (el archivo de febrero de 2024 trae `Año_de_nacimiento` corrupto), conversión de fechas e identificadores, eliminación de datos personales, filtro de duración entre 1 y 180 minutos y de duplicados, y escritura en Parquet con manifiesto.
- **P1** (`build_p1_dataset`): rejilla completa estación x día desde la apertura de cada estación (un día sin viajes es un cero real), conteo de salidas y llegadas, lags de salidas a 1, 2, 7 y 14 días, medias móviles de 7 y 28 días, lags de llegadas, calendario (día de la semana, mes, fin de semana y festivos de México, incluidos Jueves y Viernes Santo) y atributos de la estación (capacidad, coordenadas, municipio). Resultado: 358,033 filas. Entrenamiento con regresión y evaluación por estación-día y agregada por día.
- **P2** (`build_p2_dataset`): rejilla estación x bloque de 3 h, salidas, llegadas, flujo neto y etiqueta de riesgo; lags del flujo neto, de las salidas y del riesgo en el bloque anterior, el mismo bloque del día anterior y de la semana anterior; promedio de |flujo neto| del mismo bloque en las cuatro semanas previas; bloque, calendario y capacidad. Resultado: 2,167,980 filas.

- **P3** (`build_p3_dataset`): perfil de uso por estación con los viajes de entrenamiento (373 estaciones con al menos 30 salidas), estandarización y KMeans. El mismo perfil calculado con el periodo de prueba (468 estaciones) sirve para medir la estabilidad de los grupos.

La exploración que justifica estas variables está en `notebooks/1.0-dfc-exploracion-mibici.ipynb`: la correlación entre los viajes de un día y los del mismo día de la semana anterior es de 0.86, el uso entre semana tiene dos picos (07-09 h y 18-19 h) y el de fin de semana se concentra a media mañana, y hay estaciones que reciben o despachan bicicletas de forma sistemática.

5. Modelos y métricas

Tabla 3. Baseline, modelo candidato y métricas por pregunta.

	Baseline	**Candidato**	**Métricas y justificación**
P1	Naive estacional: los viajes del mismo día de la semana anterior (`departures_lag7`).	`HistGradientBoostingRegressor` con pérdida Poisson sobre lags, calendario y atributos de la estación.	**MAE** (error en viajes, interpretable para el operador), **WAPE** (error relativo al volumen total, comparable entre periodos), RMSE (penaliza errores grandes en estaciones troncales) y R^2. La pérdida Poisson se elige porque el objetivo es un conteo asimétrico.
P2	Persistencia semanal: riesgo si el mismo bloque de la semana anterior estuvo en riesgo; puntaje = |flujo neto| medio del bloque en 4 semanas / umbral.	`HistGradientBoostingClassifier` con `class_weight="balanced"`; umbral que maximiza F1 en validación.	Con 2.5 % a 5 % de positivos la exactitud no informa, así que se usa **PR-AUC** (average precision) para comparar puntajes sin fijar umbral, **recall** (estaciones en riesgo detectadas), precisión (visitas útiles del camión) y F1 en el umbral operativo; ROC-AUC como referencia (scikit-learn developers, s.f.).
P3	KMeans (k = 6) solo con latitud y longitud: zonas geográficas.	KMeans sobre los perfiles de uso estandarizados; k entre 3 y 8 elegido por silueta.	Sin etiquetas, se usan índices internos: **silueta**, Davies-Bouldin y Calinski-Harabasz, todos medidos en el espacio de perfiles de uso para que baseline y candidato se comparen en lo que importa (patrones de uso). Se añade la estabilidad temporal: ARI entre el grupo de cada estación en entrenamiento y el que recibe con su perfil de prueba.

Baselines intermedios. Para medir cuánto aporta cada nivel de complejidad, además del baseline y el candidato se evalúan modelos intermedios. En P1, la media móvil de 7 días, el promedio de entrenamiento por estación y día de la semana, y una regresión de Poisson lineal con las mismas variables del candidato. En P2, la frecuencia histórica de riesgo por estación, bloque y tipo de día, y una regresión logística balanceada. En P3, una regla fija que clasifica cada estación por la fracción de sus salidas pico de días hábiles que ocurren en la mañana (06-10 h frente a 16-20 h): más de 0.6, origen matutino; menos de 0.4, destino matutino; el resto, mixta. A

diferencia del baseline geográfico, cuyo ARI es 1.0 por construcción porque las coordenadas no cambian, la regla sí permite medir la estabilidad real de los grupos.

6. Arquitectura del sistema

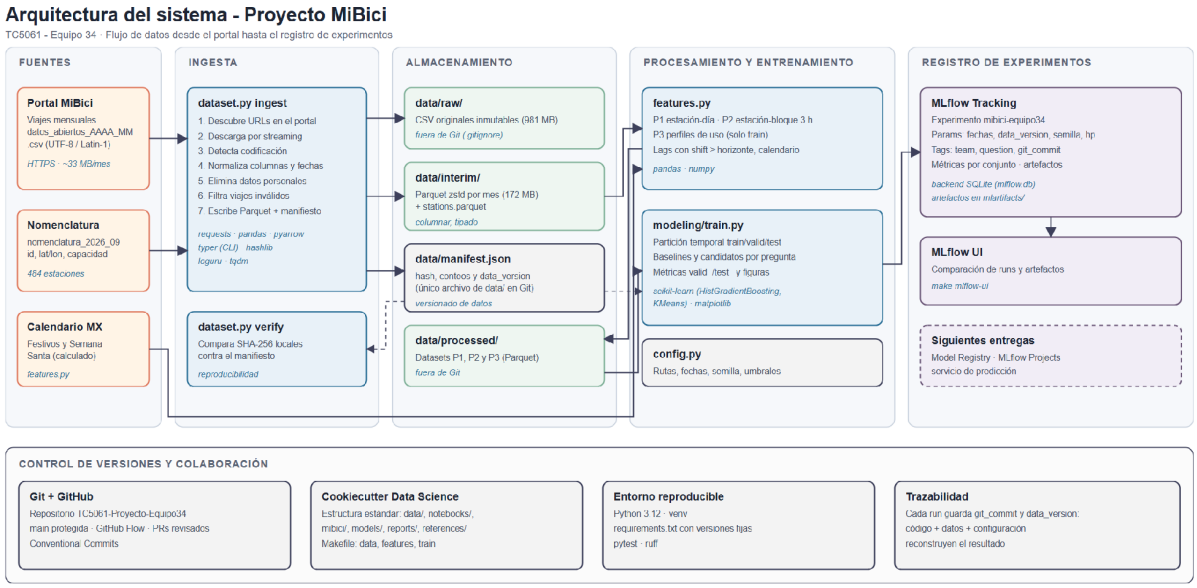


Figura 2. Componentes de la solución y flujo de datos.

Tabla 4. Herramientas por componente y justificación con las lecturas.

Componente	Herramienta	Justificación
Ingesta	`requests`, `pandas`, CLI con `typer`	La ingesta es código y no un paso manual: un comando reconstruye los mismos datos en cualquier computadora. Huyen (2022) señala que los datos de fuentes externas cambian de formato sin aviso; aquí cambian la codificación y los encabezados, y el código lo resuelve de forma repetible.
Almacenamiento	Parquet con `pyarrow` (zstd)	Formato columnar y tipado, recomendado por Huyen (2022) para cargas analíticas: reduce 981 MB de CSV a 172 MB y permite leer solo las columnas necesarias.
Versionado de datos	Manifiesto JSON con SHA-256 versionado en Git	Treveil et al. (2020) ubican la trazabilidad de datos como requisito para reproducir modelos. El manifiesto liga cada run con la versión exacta de los datos sin subir datos al repositorio; una herramienta como DVC queda como opción si crecen los volúmenes.
Procesamiento	`pandas`, `numpy` en `features.py`	Wilson (2022) recomienda mover la lógica del notebook a funciones modulares y probadas; las funciones documentan su pertinencia y evitan la fuga temporal que Huyen (2022) identifica como uno de los errores más comunes.
Entrenamiento	`scikit-learn`	API común para baselines, gradient boosting y KMeans. Huyen (2022) y Wilson (2022) recomiendan empezar con un baseline simple y justificar cada aumento de complejidad contra él.
Registro de experimentos	MLflow Tracking (backend SQLite)	Guarda parámetros, métricas, artefactos y commit de cada run para compararlos y reproducirlos (MLflow, s.f.); Wilson (2022) lo usa como herramienta central de seguimiento.
Colaboración	Git, GitHub Flow, Cookiecutter Data Science	Estructura estándar que cualquier integrante entiende, revisión por Pull Request y rama principal protegida, prácticas de colaboración que Treveil et al. (2020) proponen para escalar proyectos de ML.

7. Experimento base en MLflow

El comando `python -m mibici.modeling.train` registra doce runs en el experimento `mibici-equipo34` (cinco de P1, cuatro de P2 y tres de P3), todos ejecutados desde el código del repositorio en el commit `b11c18e` con el árbol de trabajo limpio (`git_dirty = false`). Cada run registra como parámetros el rango de fechas, la `data_version`, los cortes de partición, la semilla, la granularidad, el horizonte, el modelo y sus hiperparámetros; como etiquetas el equipo, la pregunta, el rol del modelo, el commit y la rama; métricas con el conjunto en el nombre (`valid_`, `test_`, `train_`); y al menos una gráfica y un `metrics.json` como artefactos. Las capturas del Anexo muestran la lista de runs, el detalle de un run, un artefacto y la comparación de dos runs. Al borrar la base de tracking y repetir el experimento se obtienen las mismas métricas.

Tabla 5. Resultados del experimento base.

Run	Validación	Prueba
p1-baseline-naive-estacional-7d	MAE 7.91 · WAPE 0.258	MAE 7.12 · WAPE 0.261 · R² 0.794
p1-baseline-media-movil-7d	MAE 8.27 · WAPE 0.270	MAE 7.14 · WAPE 0.262 · R² 0.838
p1-baseline-historico-estacion-dia-semana	MAE 7.42 · WAPE 0.242	MAE 8.11 · WAPE 0.298 · R² 0.711
p1-poisson-lineal-lags-calendario	MAE 6.07 · WAPE 0.198	MAE 6.00 · WAPE 0.220 · R² 0.862
p1-hgb-regressor-lags-calendario	MAE 5.42 · WAPE 0.177	MAE 5.30 · WAPE 0.194 · R² 0.893
p2-baseline-persistencia-semanal	PR-AUC 0.401 · F1 0.342	PR-AUC 0.387 · F1 0.350 · recall 0.351
p2-baseline-tasa-historica-estacion-bloque	PR-AUC 0.383 · F1 0.390	PR-AUC 0.320 · F1 0.346 · recall 0.400
p2-logistica-flujos-lag	PR-AUC 0.409 · F1 0.413	PR-AUC 0.405 · F1 0.415 · recall 0.450
p2-hgb-classifier-flujos-lag	PR-AUC 0.485 · F1 0.459	PR-AUC 0.464 · F1 0.449 · recall 0.448
p3-baseline-kmeans-geografico (k = 6)	silueta en perfiles -0.014 · Davies-Bouldin 4.48	
p3-baseline-regla-manana-tarde (3 grupos)	silueta en perfiles 0.051 · Davies-Bouldin 3.17 · ARI de estabilidad 0.271	
p3-kmeans-perfiles-uso (k = 3)	silueta en perfiles 0.142 · Davies-Bouldin 2.03 · ARI de estabilidad 0.446	

7.1. Ampliación de la búsqueda de hiperparámetros

Después de ejecutar el experimento base, se amplió la exploración de modelos mediante nuevas ejecuciones registradas en MLflow. El objetivo fue evaluar si el ajuste de hiperparámetros permitía mejorar el desempeño de los candidatos iniciales y documentar de manera reproducible el proceso de selección.

Para P1, se exploraron distintos valores de alpha para la regresión de Poisson y combinaciones de learning_rate y max_leaf_nodes para HistGradientBoostingRegressor. Para P2, se evaluaron valores de C para la regresión logística y combinaciones de learning_rate y max_leaf_nodes para HistGradientBoostingClassifier. Para P3, se amplió la búsqueda de KMeans a valores de (k) entre 3 y 8 y diferentes valores de n_init.

Las ejecuciones se registraron en el experimento mibici-equipo34, conservando los identificadores, parámetros y métricas disponibles en MLflow. Esto permite comparar los candidatos con los modelos del experimento base y mantener la trazabilidad de los resultados.

7.2. Comparación de candidatos y selección final

La Tabla 6 resume algunos de los resultados obtenidos durante la búsqueda ampliada. La selección final debe basarse en las métricas de validación y en los criterios definidos para cada pregunta; las métricas de prueba se utilizan para evaluar el desempeño final de los candidatos seleccionados.

Tabla 6:

Pregunta	Candidato	Métricas de validación	Métricas de prueba
P1	HGBRegressor original	WAPE = 0.176731	WAPE = 0.194318; MAE = 5.299791
P1	HGBRegressor (learning_rate=0.03, max_leaf_nodes=31)	WAPE = 0.178650	WAPE = 0.193537; MAE = 5.278495
P2	HGBClassifier original	PR-AUC = 0.482608	PR-AUC = 0.464254
P2	HGBClassifier (learning_rate=0.1, max_leaf_nodes=31)	PR-AUC = 0.482085	PR-AUC = 0.463514
P3	KMeans (k=3)	Silueta de perfiles = 0.142346	ARI de estabilidad = 0.446432

Los resultados muestran que el ajuste de hiperparámetros no garantiza una mejora en todas las métricas. En P1, la nueva combinación reduce ligeramente el error de prueba, pero obtiene un WAPE de validación superior al del candidato original. En P2, la combinación evaluada no supera el PR-AUC de prueba del HGBClassifier original. Por ello, estos resultados no justifican por sí solos sustituir los candidatos iniciales.

En P3, KMeans con tres grupos mantiene una silueta de perfiles de aproximadamente 0.142. La decisión de conservar esta configuración debe considerar también la separación entre grupos, la estabilidad y la utilidad de los perfiles obtenidos.

La experimentación ampliada permite documentar las alternativas evaluadas y justificar las decisiones de selección a partir de resultados registrados, en lugar de elegir un modelo únicamente por su configuración o complejidad.

P1. El candidato reduce el MAE en prueba de 7.12 a 5.30 viajes por estación-día (25.6 %) y el WAPE de 26.1 % a 19.4 %. La Figura 3 muestra que sigue el nivel y la estacionalidad semanal del sistema. Los días atípicos (lluvias, eventos) todavía no tienen una variable que los explique,

por lo que los datos de clima son la siguiente mejora natural. La regresión de Poisson lineal, con las mismas variables, ya baja el MAE a 6.00: buena parte de la mejora viene de las variables y el gradient boosting agrega el resto. El promedio histórico por estación y día de la semana es el mejor baseline en validación (MAE 7.42) y el peor en prueba (8.11); es consistente con un cambio en la demanda de 2026, aunque también pueden influir las estaciones nuevas, que no tienen historia, y que la prueba abarca el verano.

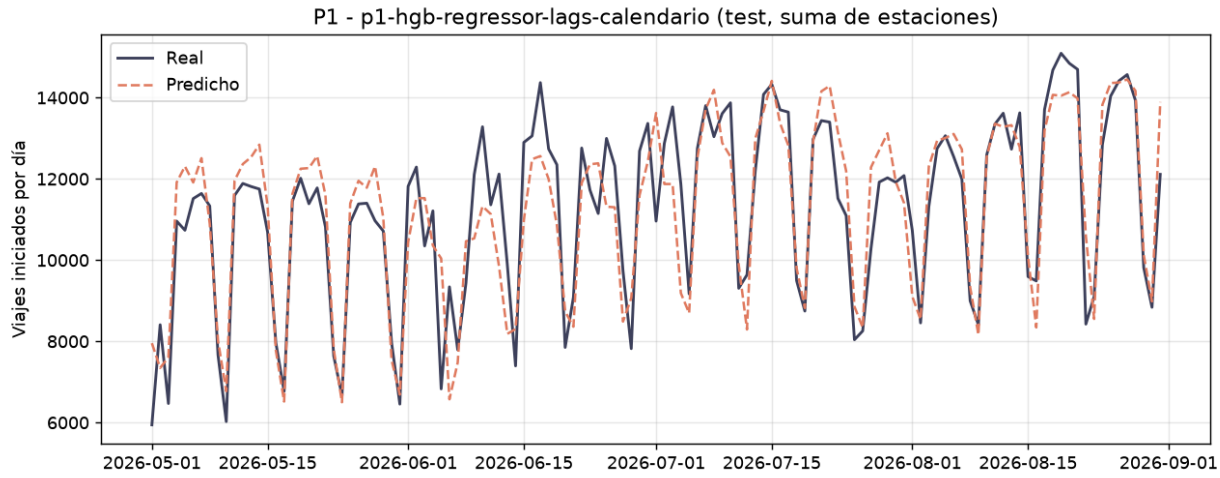


Figura 3. P1: viajes diarios del sistema en prueba, reales contra predichos por el candidato.

P2. El candidato sube el PR-AUC de 0.387 a 0.464 con una prevalencia de 2.5 % en prueba (un clasificador aleatorio tendría 0.025) y detecta 44.8 % de los bloques en riesgo con una precisión de 45 %: de cada dos estaciones que señala, casi una está realmente en riesgo. El umbral se fijó en validación (0.868) y se aplicó sin cambios en prueba, donde precisión y recall quedan muy cerca de los de validación. Un umbral tan alto, frente a esa prevalencia, refleja que con `class_weight="balanced"` los puntajes no son probabilidades calibradas: en prueba la probabilidad media predicha es 0.17. Sirven para priorizar estaciones, no para comunicar un porcentaje de riesgo (van den Goorbergh et al., 2022). La prevalencia baja de 5.0 % en entrenamiento a 2.5 % en prueba, posiblemente por la llegada de estaciones nuevas, y es una deriva que se vigilará. La regresión logística queda entre ambos modelos (PR-AUC de 0.405) y la tasa histórica de riesgo, por debajo del baseline (0.320).

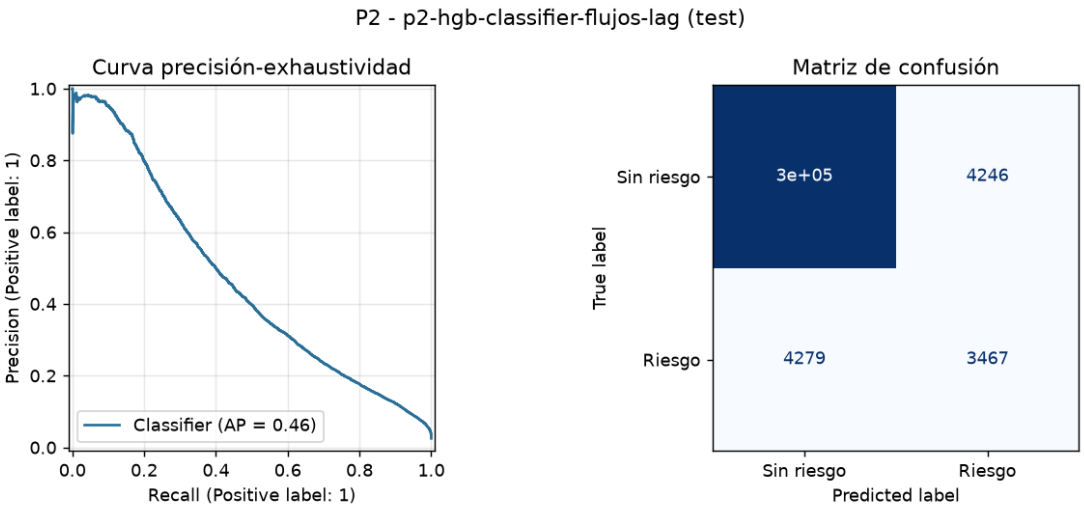


Figura 4. P2: curva precisión-exhaustividad y matriz de confusión del candidato en prueba.

P3. Agrupar solo por geografía produce zonas compactas (silueta de 0.47 en su propio espacio), pero en el espacio de patrones de uso su silueta es negativa: estaciones vecinas no necesariamente se usan igual. El candidato encuentra tres grupos (Figura 5): un grupo de 172 estaciones con pico matutino de salidas, típico de zonas residenciales de origen; otro de 173 con pico vespertino, típico de destinos laborales y comerciales; y un grupo de 28 estaciones con uso concentrado en la media mañana del fin de semana, un patrón que sugiere uso recreativo. La silueta de 0.14 indica grupos con traslape, y un ARI de 0.45 indica que la asignación es moderadamente estable al recalcular los perfiles con los meses de prueba. La regla fija de mañana y tarde obtiene una silueta de 0.051 y un ARI de 0.271, así que KMeans sí captura estructura más allá de esa división obvia. Afinar k y las variables es trabajo de la siguiente entrega.

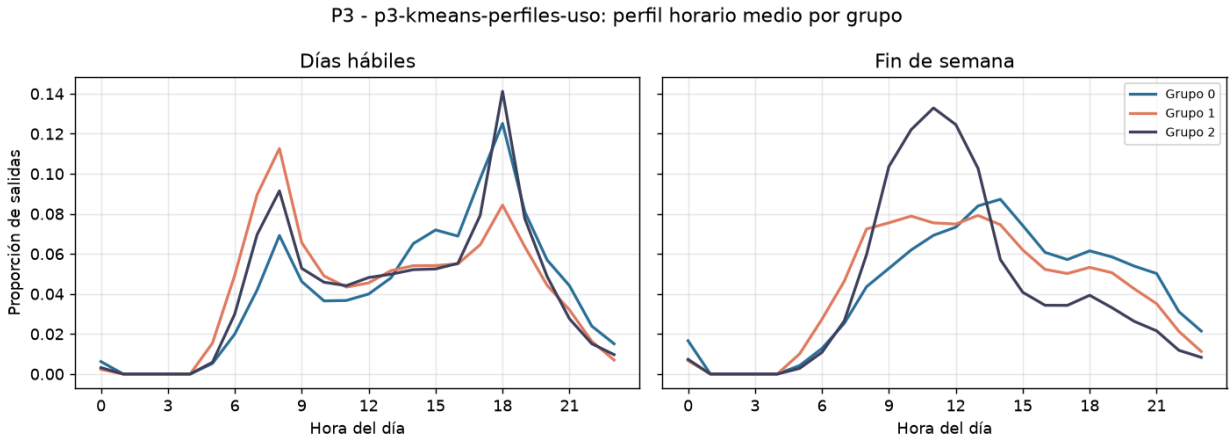


Figura 5. P3: perfil horario medio de salidas de cada grupo del candidato.

7.3. Ciclo de vida de los modelos y justificación

El ciclo de vida de los modelos se organizó para mantener la trazabilidad desde la preparación de los datos hasta el registro de los candidatos seleccionados. El proceso comienza con la ingesta y preparación de los datos de MiBici, continúa con la construcción de variables y la separación de los conjuntos de entrenamiento, validación y prueba, y posteriormente ejecuta los modelos base y las búsquedas de hiperparámetros.

Las ejecuciones se registran en MLflow con parámetros, métricas, etiquetas y artefactos que permiten comparar los resultados. La selección se realiza a partir de las métricas de validación definidas para cada pregunta. El conjunto de prueba se utiliza para evaluar el desempeño final sin utilizarlo como criterio para ajustar los hiperparámetros.

Una vez seleccionados los candidatos, estos se registran en MLflow Model Registry para facilitar su identificación, carga y seguimiento mediante sus versiones y ejecuciones de origen.

El flujo se justifica por tres razones: reproducibilidad, porque permite consultar los parámetros y resultados de cada ejecucion; comparabilidad, porque centraliza las métricas de diferentes modelos en un mismo experimento; y trazabilidad, porque relaciona los modelos registrados con las ejecuciones que los generaron. Este enfoque facilita revisar las decisiones tomadas y repetir los experimentos cuando se incorporan nuevas variables o se modifican los modelos.

8. Estructura del repositorio y flujo de trabajo

La estructura se generó con `pipx install cookiecutter-data-science` y `ccds` con `include_code_scaffold = Yes`, entorno virtualenv, `requirements.txt`, pytest y ruff, y se subió como primer commit. Sobre ella, rutas, fechas, semilla y umbrales viven solo en `mibici/config.py`; las dependencias tienen versión fija; el `.gitignore` excluye los datos de MiBici (salvo `data/manifest.json`), `mlruns/`, `mlartifacts/`, `mlflow.db`, modelos serializados y archivos `.env`. El historial no contiene datos, modelos, carpetas de MLflow ni credenciales.

```
diegofalconcostilla/TC5061-Proyecto-Equipo34 — git ls-files
TC5061-Proyecto-Equipo34/
├── data/
│   └── manifest.json
├── mibici/
│   ├── modeling/
│   │   ├── __init__.py
│   │   ├── predict.py
│   │   └── train.py
│   ├── __init__.py
│   ├── config.py
│   ├── dataset.py
│   ├── features.py
│   └── plots.py
├── notebooks/
│   └── 1.0-dfc-exploracion-mibici.ipynb
├── references/
│   ├── arquitectura.md
│   ├── arquitectura.png
│   ├── ml_canvas.md
│   └── ml_canvas.png
├── reports/
│   └── figures/
│       └── mlflow/
│           ├── 01_lista_runs.png
│           ├── 02_detalle_run.png
│           ├── 03_artefacto.png
│           └── 04_comparacion.png
├── tests/
│   ├── test_dataset.py
│   └── test_features.py
├── .gitignore
├── LICENSE
├── Makefile
├── pyproject.toml
├── README.md
└── requirements.txt
```

Figura 6. Archivos versionados en el repositorio.

El equipo sigue GitHub Flow: `main` está protegida (requiere Pull Request, no admite push directo ni force push, también para administradores) y cada cambio se desarrolla en una rama `<tipo>/<descripcion>` con commits en formato Conventional Commits (`feat(dataset): ...`, `docs: ...`). La base del proyecto entró en cinco Pull Requests encadenados: configuración, ingesta, features, experimento y documentación. Cada integrante abre al menos un Pull Request y revisa al menos uno de otro integrante.

9. Siguientes pasos

- Agregar variables de clima y eventos para P1 y P2, y probar horizontes de varios días.
- Calibrar la aproximación de riesgo de P2 con distintos umbrales y con una estimación de ocupación acumulada.

- Explorar k y otros algoritmos de clustering (jerárquico, DBSCAN) para P3.
- Registrar los modelos en el MLflow Model Registry y empaquetar el entrenamiento como MLflow Project.
- Calibrar las probabilidades de P2 con el periodo de validación si se usarán para decisiones basadas en costos.
- Tratar explícitamente las estaciones nuevas, que solo aparecen en validación y prueba, y vigilar el cambio en la prevalencia del riesgo.

Referencias

Cookiecutter Data Science. (s.f.). *Cookiecutter Data Science* [Documentación]. DrivenData. https://cookiecutter-data-science.drivendata.org/

Conventional Commits. (s.f.). *Conventional Commits 1.0.0*. https://www.conventionalcommits.org/es/v1.0.0/

Dorard, L. (2018, 9 de marzo). *From data to AI with the Machine Learning Canvas (Part I)*. Own Machine Learning. Medium.

GitHub. (s.f.). *GitHub flow* [Documentación]. https://docs.github.com/en/get-started/using-github/github-flow

Huyen, C. (2022). *Designing machine learning systems: An iterative process for production-ready applications*. O'Reilly Media.

MiBici. (s.f.-a). *Aviso de privacidad*. Sistema de Bicicleta Pública del Área Metropolitana de Guadalajara. https://www.mibici.net/es/aviso-de-privacidad/

MiBici. (s.f.-b). *Datos abiertos* [Conjunto de datos]. Sistema de Bicicleta Pública del Área Metropolitana de Guadalajara. https://www.mibici.net/es/datos-abiertos/

MLflow. (s.f.). *MLflow Tracking* [Documentación]. https://mlflow.org/docs/latest/ml/tracking/

pandas development team. (s.f.). *pandas.DataFrame.to_parquet* [Documentación]. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_parquet.html

scikit-learn developers. (s.f.). *Metrics and scoring: Quantifying the quality of predictions* [Documentación]. https://scikit-learn.org/stable/modules/model_evaluation.html

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., & Heidmann, L. (2020). *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media.

van den Goorbergh, R., van Smeden, M., Timmerman, D., & Van Calster, B. (2022). The harm of class imbalance corrections for risk prediction models: Illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association, 29*(9), 1525–1534. https://doi.org/10.1093/jamia/ocac093

Wilson, B. (2022). *Machine learning engineering in action*. Manning Publications.

Anexo: evidencias de la interfaz de MLflow

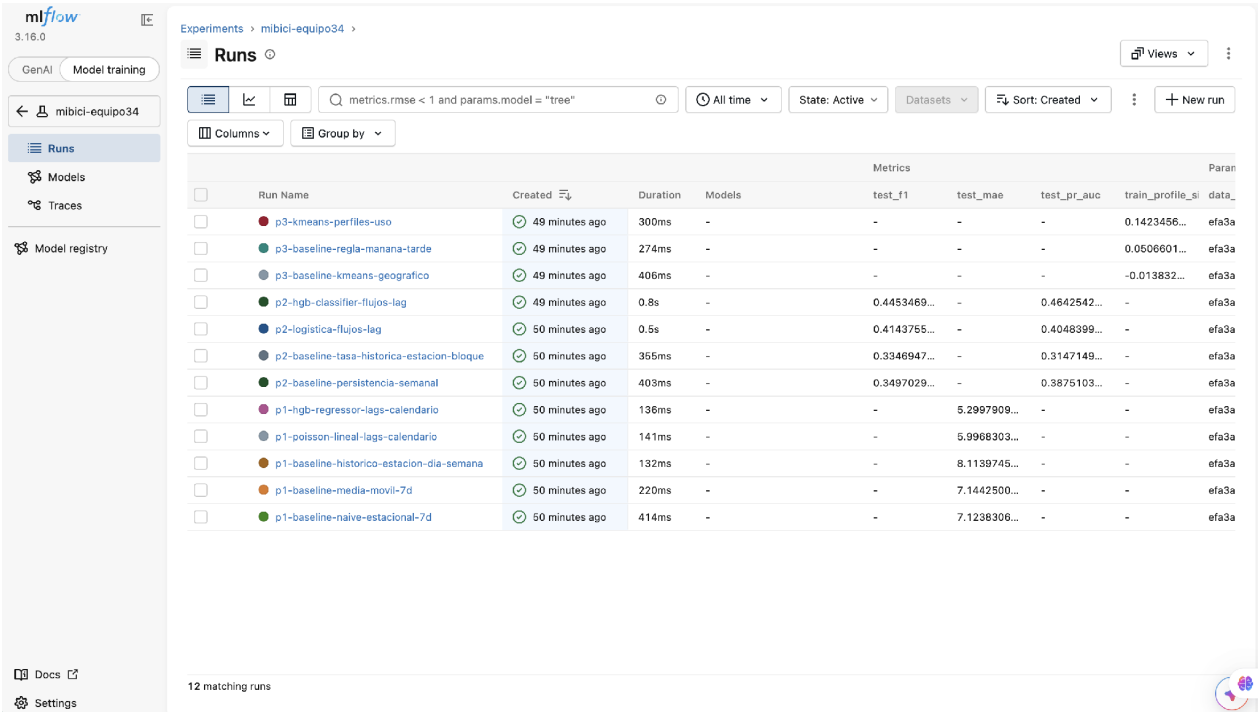


Figura A1. Lista de runs del experimento `mibici-equipo34` con métricas, versión de datos y etiquetas de pregunta y rol.

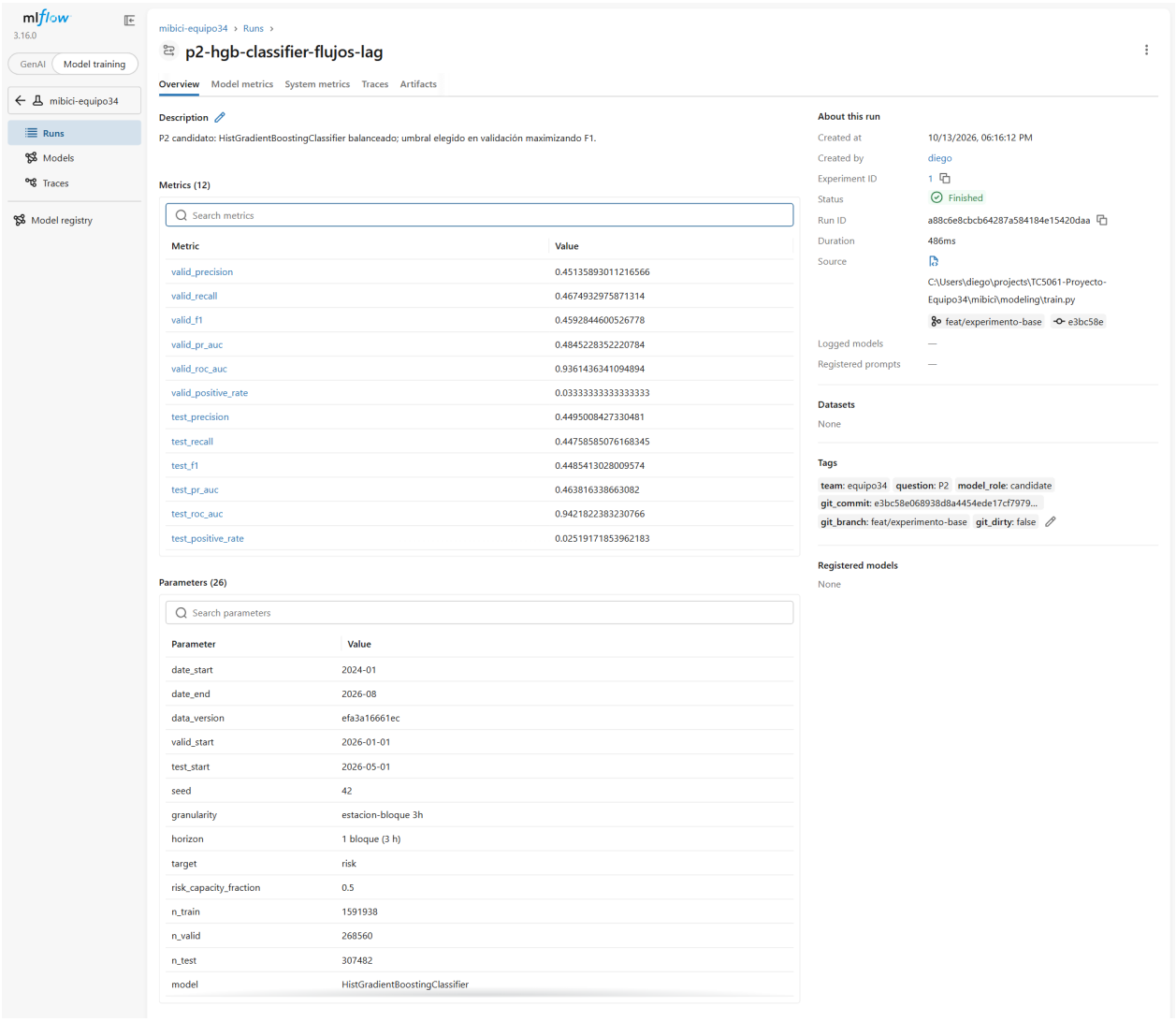


Figura A2. Detalle del run `p2-hgb-classifier-flujos-lag`: métricas, parámetros, etiquetas y commit de Git.

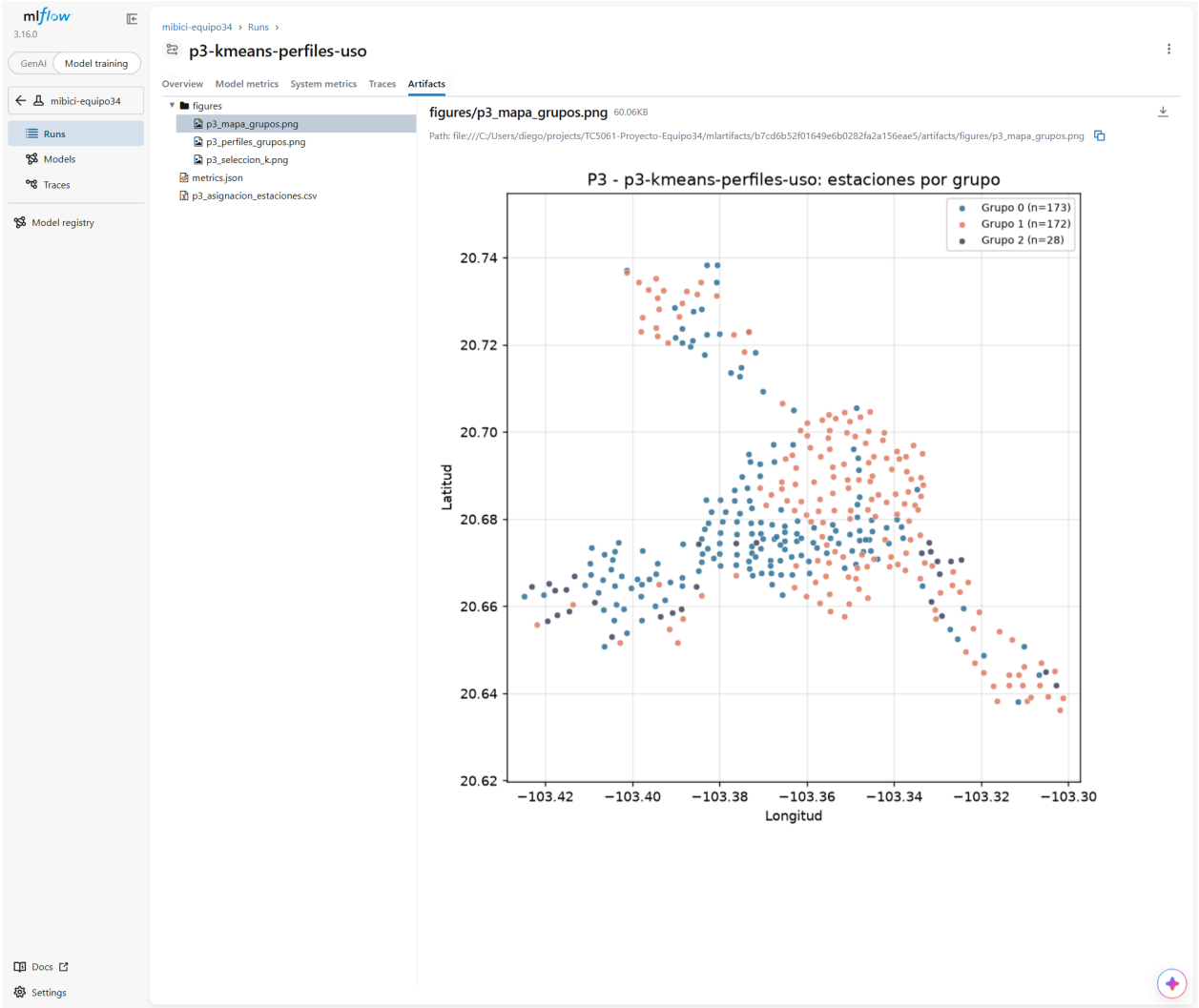


Figura A3. Artefacto registrado: mapa de los grupos de estaciones del run `p3-kmeans-perfiles-uso`.

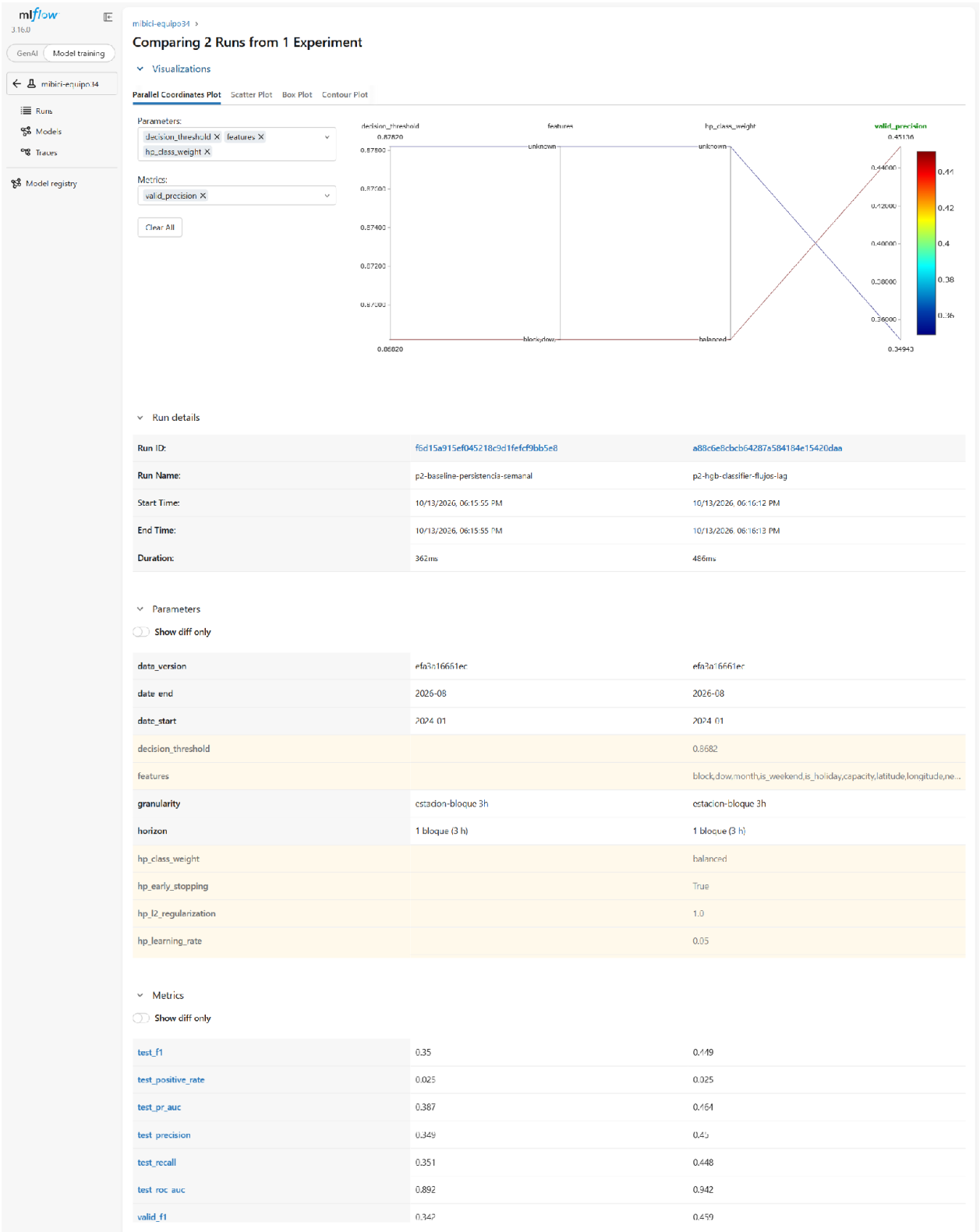


Figura A4. Comparación del baseline y el candidato de P2: parámetros y métricas lado a lado.

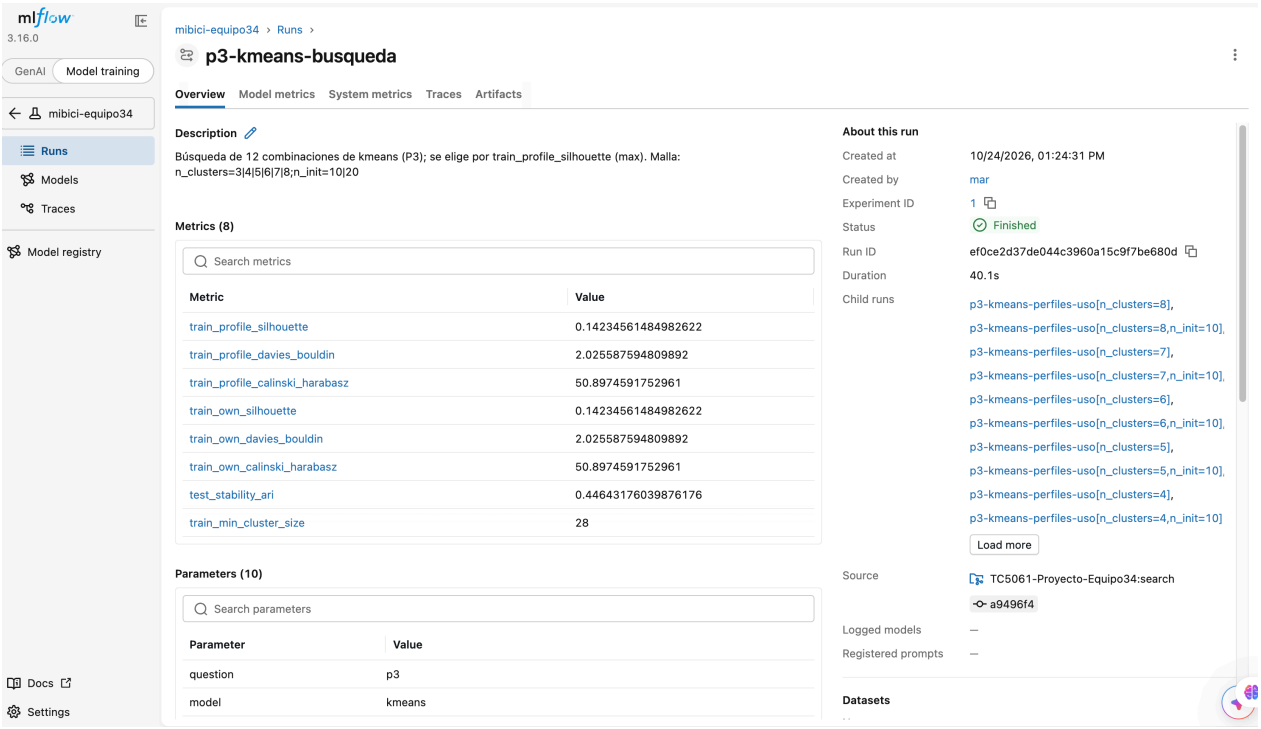


Figura A5. Ejecución principal y ejecuciones anidadas de la búsqueda de hiperparámetros de P3 en MLflow.

La interfaz muestra la ejecución principal p3-kmeans-busqueda, responsable de evaluar distintas configuraciones de KMeans, y las corridas hijas asociadas a diferentes valores de (k) y n_init. MLflow registra las métricas de calidad y estabilidad de los clústeres, lo que permite comparar las configuraciones y mantener la trazabilidad de la búsqueda.

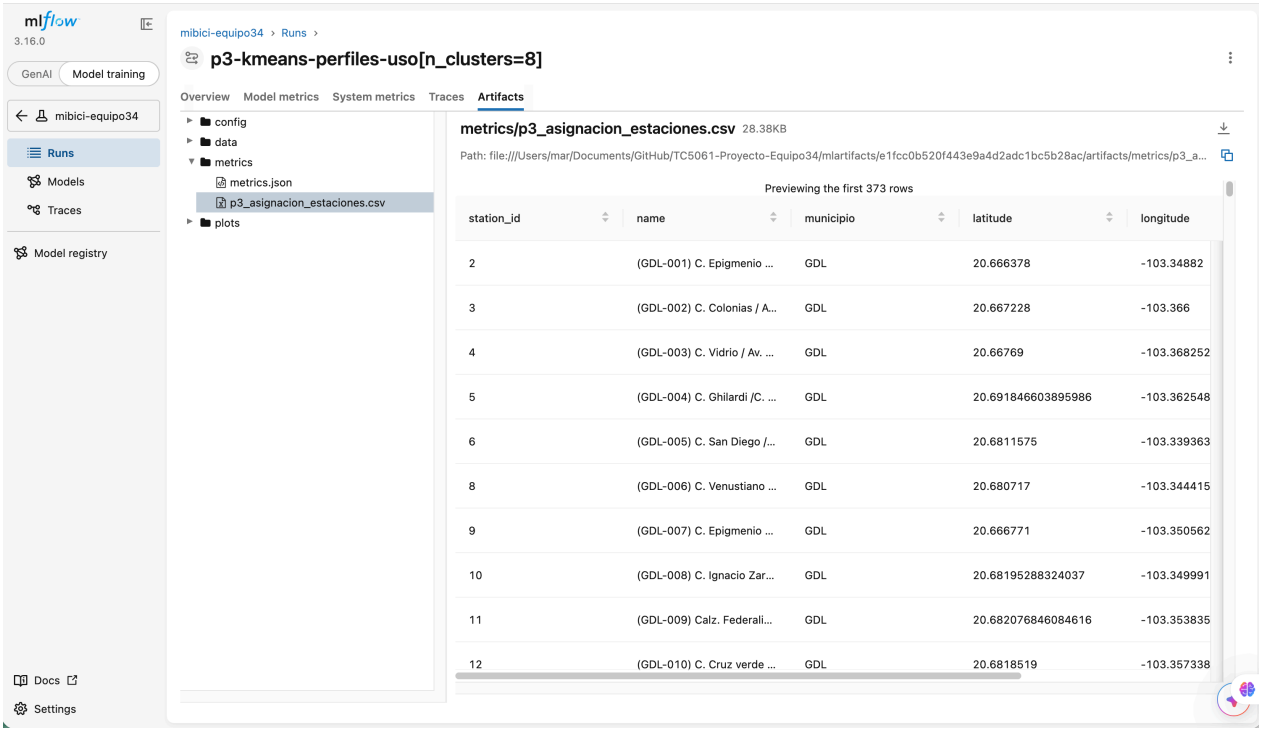


Figura A6. Artefactos registrados en una corrida candidata de P3 en MLflow. La vista de artefactos de la corrida p3-kmeans-perfiles-uso[n_clusters=8] muestra el archivo metrics.json y el archivo p3_asignacion_estaciones.csv, que contiene la asignación de las estaciones a los clústeres junto con sus atributos identificativos y geográficos. Esta evidencia permite verificar que la corrida conserva resultados exportables para su inspección y análisis posterior.

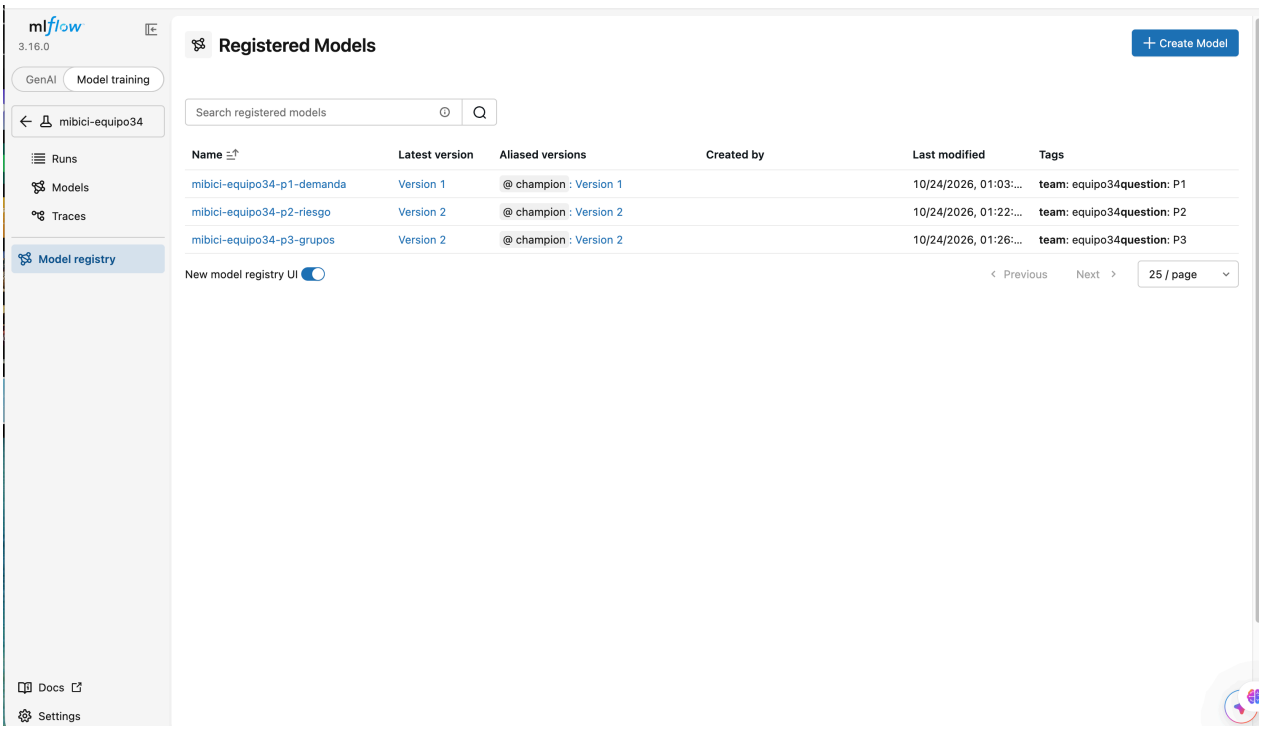


Figura A7. Modelos registrados en MLflow Model Registry. La interfaz muestra los tres modelos correspondientes a las preguntas P1, P2 y P3, con sus respectivas versiones y el alias champion. Este registro permite identificar las versiones seleccionadas y acceder a su información para facilitar su seguimiento y reutilización.

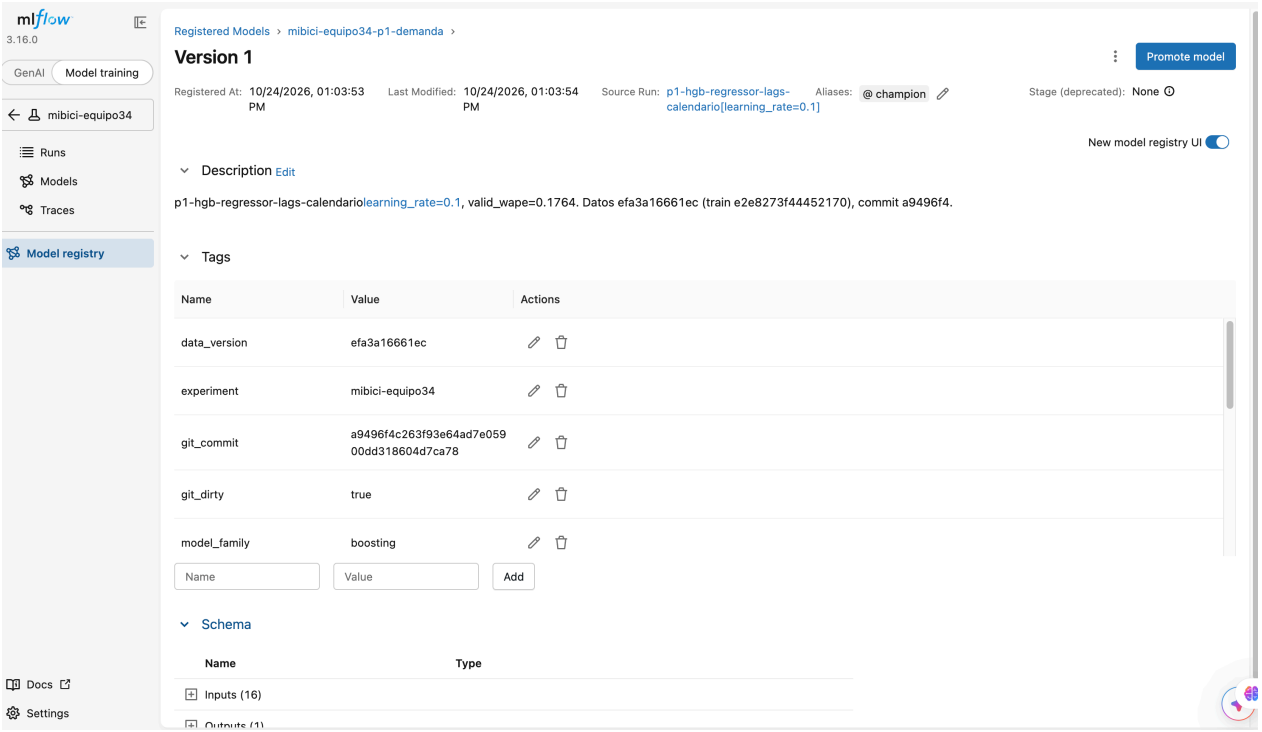


Figura A8. Linaje y metadatos de la versión registrada del modelo P1 en MLflow Model

Registry. El detalle de la versión 1 del modelo mibici-equipo34-p1-demanda identifica la corrida de origen, el alias champion, la métrica de validación y la versión de los datos utilizada. También presenta las etiquetas de trazabilidad y el esquema de entradas y salidas, facilitando la identificación de la configuración asociada al modelo registrado.

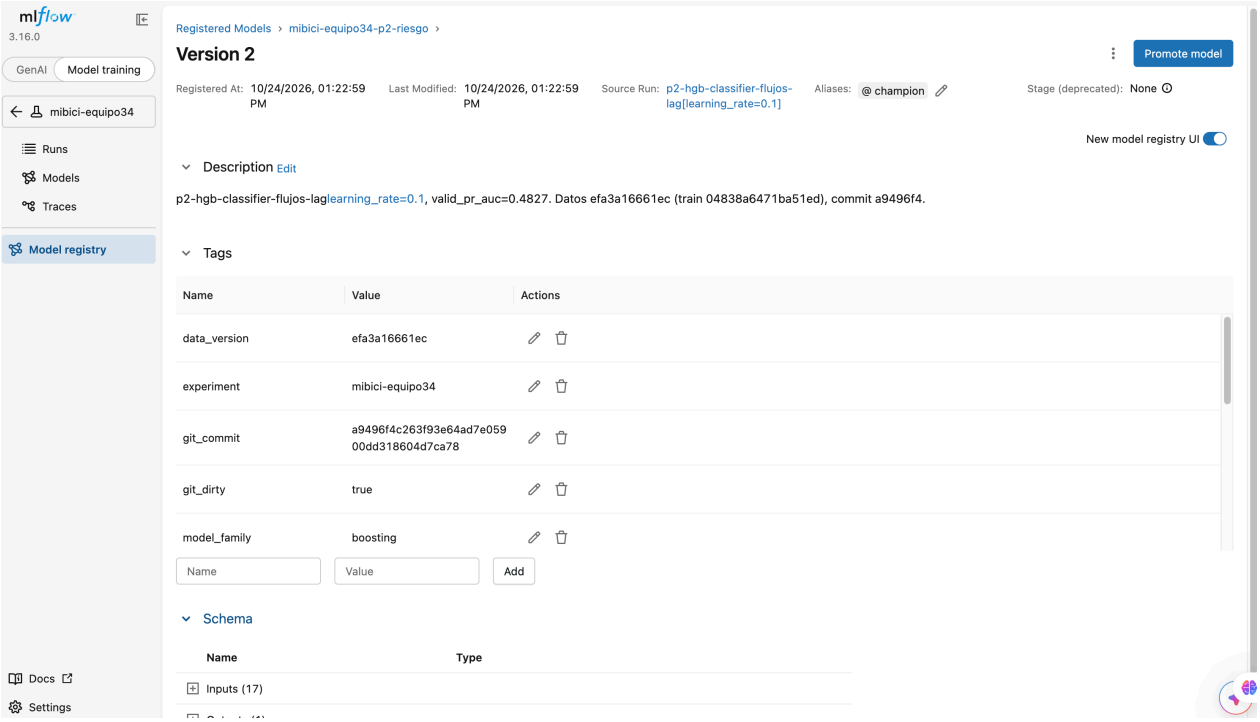


Figura A9. Linaje y metadatos de la versión registrada del modelo P2 en MLflow Model Registry. La versión 2 del modelo mibici-equipo34-p2-riesgo está asociada al alias champion y a la corrida p2-hgb-classifier-flujos-lag[learning_rate=0.1]. La interfaz muestra la métrica de validación PR-AUC, la versión de los datos, el identificador del commit y el esquema de entradas, lo que facilita la trazabilidad de la versión registrada.

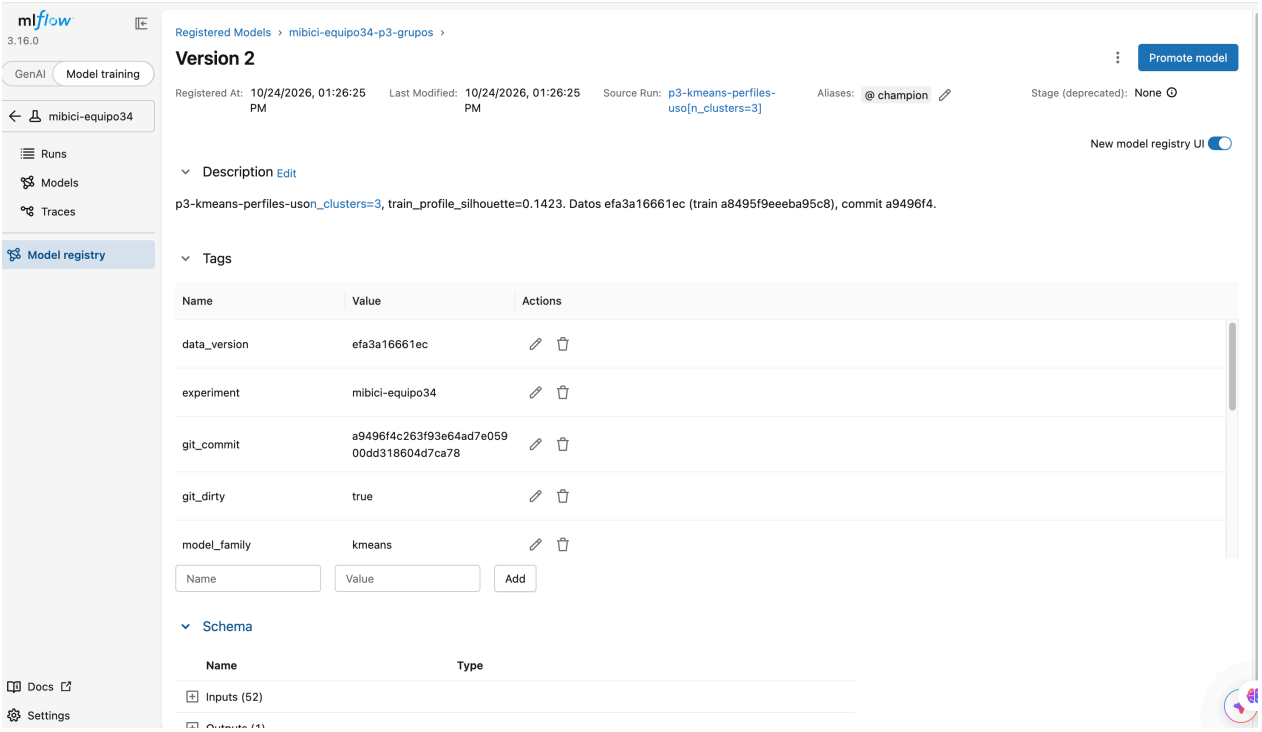


Figura A10. Linaje y metadatos de la versión registrada del modelo P3 en MLflow Model Registry. La versión 2 del modelo mibici-equipo34-p3-grupos está asociada al alias champion y a la corrida p3-kmeans-perfiles-uso[n_clusters=3]. La interfaz presenta la métrica de silueta de perfiles, la versión de los datos, el identificador del commit y el esquema de entradas del modelo, permitiendo identificar la configuración registrada y su procedencia.